

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Вычислительная математика»
Тема: МЕТОД БИСЕКЦИИ, МЕТОД ХОРД, МЕТОД НЬЮТОНА , МЕТОД
ПРОСТЫХ ИТЕРАЦИЙ

Студент гр. 4341

Сердюков Ф.С.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Метод бисекции

Цель работы.

Целью лабораторной работы являются изучение метода бисекции с заданной точностью, написание программы, реализующей данный метод, и исследование обусловленности метода (чувствительности к ошибкам в исходных данных).

Задание.

В данной лабораторной работе требуется найти корень уравнения $f(x) = 0$ методом бисекции с заданной точностью Eps , исследовать зависимость числа итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных).

Выполнение работы осуществляется по индивидуальным вариантам заданий (нелинейных уравнений), приведенным в подразделе 3.6. Номер варианта для каждого студента определяется преподавателем.

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки $[Left, Right]$, на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).

2) Составить подпрограмму вычисления функции $f(x)$.

3) Составить головную программу, содержащую обращение к подпрограмме $f(x)$, *BISECT*, *Round* и индикацию результатов.

4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps .

5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы *Round*, округляющей значения функции с заданной точностью $Delta$.

Вариант 18: $f(x) = e^{\frac{1}{x^2}} - \ln x$

Отделение корней уравнения.

Рассмотрим функцию $f(x) = e^{\frac{1}{x^2}} - \ln x$ и её поведение. Она определена и непрерывна на $(0, +\infty)$. Её производная равна:

$$f'(x) = \frac{-2e^{\frac{1}{x^2}}}{x^3} - \frac{1}{x}$$

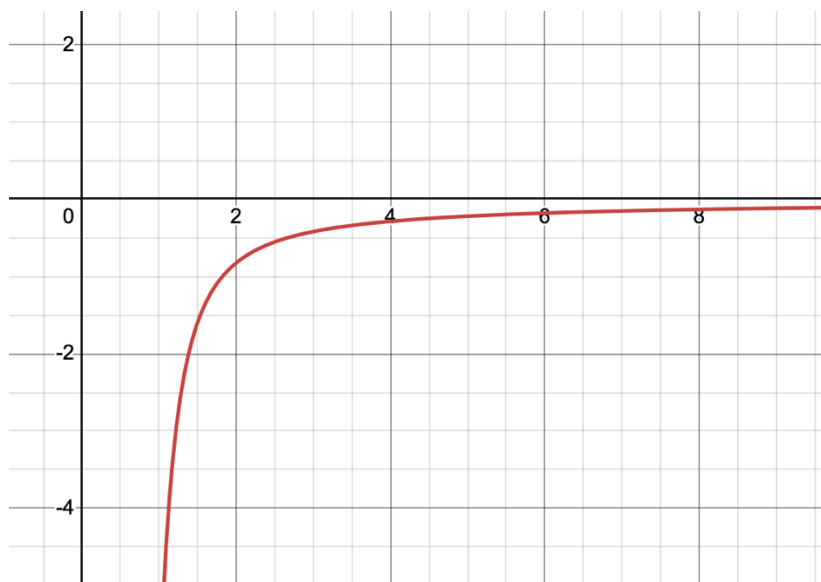


Рисунок 1 – График $f'(x)$

$f'(x) < 0$ для всех $x > 0$. Это значит, что функция монотонно убывает на всей области определения.

При $x \rightarrow 0^+$: $-\ln x \rightarrow +\infty$, $e^{\frac{1}{x^2}} \rightarrow +\infty$, поэтому $f(x) \rightarrow +\infty$.

При $x \rightarrow +\infty$: $-\ln x \rightarrow -\infty$, $e^{\frac{1}{x^2}} \rightarrow 1$, поэтому $f(x) \rightarrow -\infty$.

Функция непрерывна на $(0, +\infty)$ и меняет знак с $+\infty$ на $-\infty$. По теореме Больцано-Коши корень существует. Так как функция строго убывает, корень единственный.

Локализуем корень:

Найдем значения функции в некоторых точках, чтобы определить отрезок для метода бисекции:

Локализуем корень:

При $x = 1$: $f(1) = e^1 - \ln 1 = e - 0 \approx 2.718 > 0$

При $x = 3$: $f(3) = e^{(1/9)} - \ln 3 \approx 1.117 - 1.099 \approx 0.018 > 0$

При $x = 4$: $f(4) = e^{(1/16)} - \ln 4 \approx 1.064 - 1.386 \approx -0.322 < 0$

Для метода бисекции будет взят отрезок $[3, 4]$, так как на его концах функция принимает значения разных знаков.

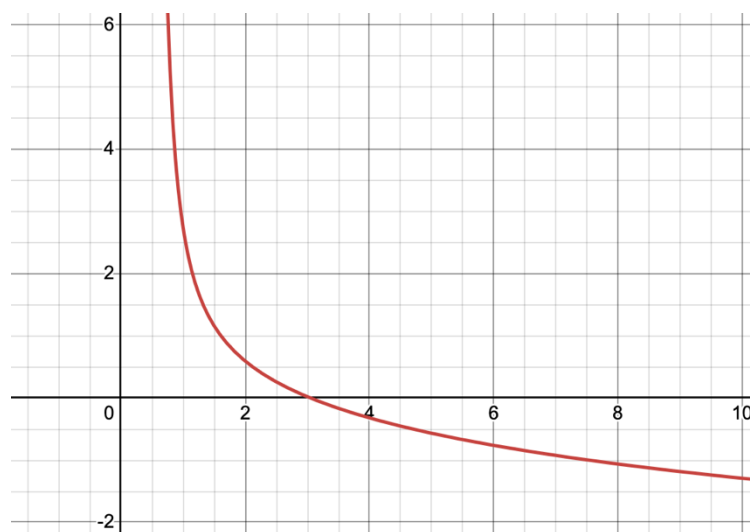


Рисунок 2 – График $f(x)$

Разработка программы.

Программа, реализующая метод бисекции, была написана на языке программирования Python

Разработанный программный код см. в **приложении А**.

Исследование зависимости числа итераций от эпсилон.

Для подсчета числа итераций в функции *bisect* возвращался счетчик количества итераций. При каждой итерации значение счётчика увеличивалось на 1.

Ниже приведена гистограмма теоретической и практической зависимости числа итераций от *Eps* по формуле $N \approx \log_2 \frac{b-a}{Eps}$ (см. рис. 2).

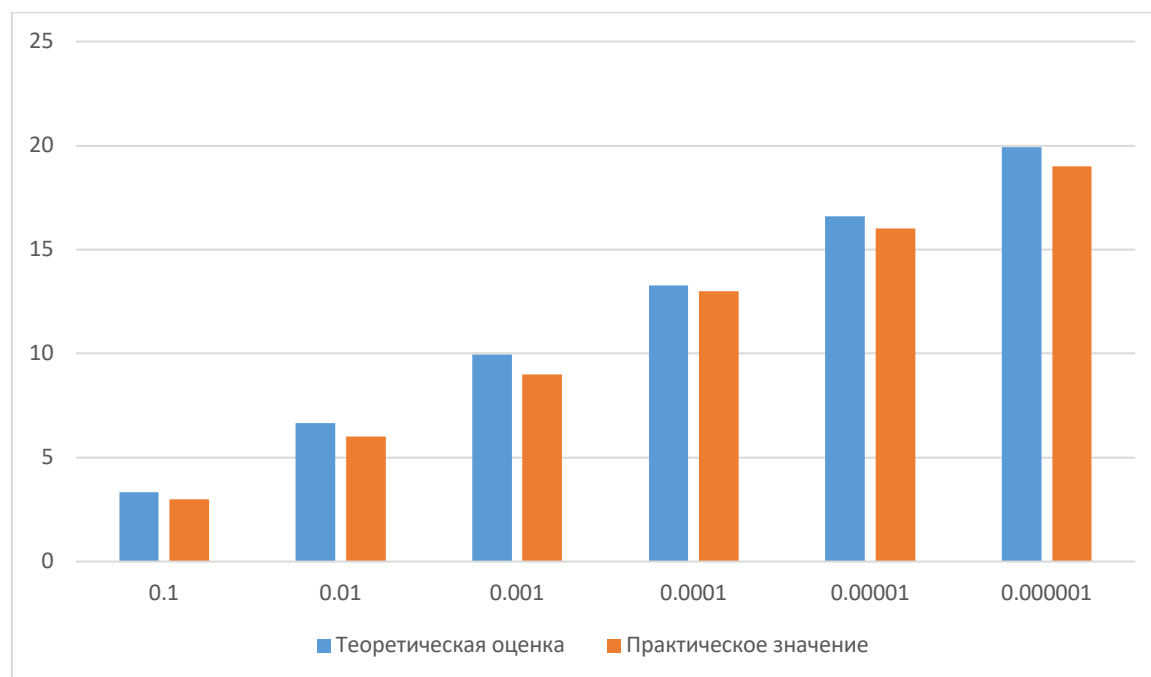


Рисунок 3 – Зависимость числа итераций от *Eps*

Из графика видно, что полученное на практике значение близко к теоретической оценке количества итераций.

Тестирование.

В табл. 1 приведены результаты работы программы для различных значений эпсилон.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные
1.	0.1	3.1
2.	0.01	3.04
3.	0.000001	3.045936

Чувствительность метода к ошибкам в исходных данных.

Для анализа чувствительности метода к ошибкам в исходных данных код был дополнен при помощи функции *def round_value(x, delta)*, которая округляла значения функции $f(x)$ с заданной точностью *delta* для симуляции ошибок в данных.

Для каждого значения *Eps* перебиралось несколько значений *delta*, чтобы установить чувствительность метода к исходным данным. Полученные результаты бисекции показаны в табл. 2.

Исследование чувствительности метода к ошибкам в данных

Введите требуемую точность (например, 0.001):

0.1

Найденный корень: 3.1

=====			
Eps	Корень	Итер.	

0.1	3.0625000000	3	
0.01	3.0390625000	6	
0.001	3.0458984375	9	
0.0001	3.0459594727	13	
0.00001	3.0459365845	16	
0.000001	3.0459356308	19	

Исследование чувствительности к округлению (delta):

(при фиксированном Eps = 0.0001)

Delta	Корень	

0.1	3.0000000000	
0.01	3.0468750000	
0.001	3.0468750000	
0.0001	3.0458984375	
0.00001	3.0459594727	
0.000001	3.0459594727	

Введите требуемую точность (например, 0.001):

0.01

Найденный корень: 3.04

Eps	Корень	Итер.
0.1	3.0625000000	3
0.01	3.0390625000	6
0.001	3.0458984375	9
0.0001	3.0459594727	13
0.00001	3.0459365845	16
0.000001	3.0459356308	19

Исследование чувствительности к округлению (delta):
(при фиксированном Eps = 0.0001)

Delta	Корень
0.1	3.0000000000
0.01	3.0468750000
0.001	3.0468750000
0.0001	3.0458984375
0.00001	3.0459594727
0.000001	3.0459594727

Введите требуемую точность (например, 0.001):
0.000001

Найденный корень: 3.045936

Eps	Корень	Итер.
0.1	3.0625000000	3
0.01	3.0390625000	6
0.001	3.0458984375	9
0.0001	3.0459594727	13
0.00001	3.0459365845	16
0.000001	3.0459356308	19

Исследование чувствительности к округлению (delta):
(при фиксированном Eps = 0.0001)

Delta	Корень
0.1	3.0000000000
0.01	3.0468750000
0.001	3.0468750000
0.0001	3.0458984375

0.00001	3.0459594727	
0.000001	3.0459594727	

Считая значение 3.0459361, полученное без симуляции ошибок и с ϵ равным $1e-7$, эталонным и наиболее близким к действительному, мы видим, что при уменьшении delta уменьшается и ошибка в результате.

Можем заметить, что при небольших ошибках в исходных данных, ошибка в результатах метода относительно мала: при $\delta = 0.001$ ошибка появляется в четвёртом знаке после запятой, при 0.0001 — в пятом, при 0.00001 — в шестом.

Таким образом, метод устойчив к ошибкам во входных данных и выдает относительно точные результаты.

Выводы.

В результате выполнения лабораторной работы был исследован метод бисекции. Была написана программа на языке Python, реализующая данный метод. Была исследована зависимость количества итераций, реализуемых методом, от значения требуемой точности. Полученная практическая оценка количества итераций близка к теоретической.

Также была исследована чувствительность метода бисекции к ошибкам во входных данных. Было установлено, что метод устойчив к ошибкам в исходных данных и выдает контролируемо малую погрешность в выходных данных.

Метод хорд

Цель работы.

Целью лабораторной работы являются изучение метода хорд с заданной точностью, написание программы, реализующей данный метод, исследование скорости сходимости и обусловленности метода (чувствительности к ошибкам в исходных данных).

Задание.

лабораторной работе №4 предлагается, используя программы - функции HORDA и Round из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения $f(x) = 0$ с заданной точностью Eps методом хорд, исследовать скорость сходимости и обусловленности метода.

Для данной работы, как и для лабораторной работы №3 задаются индивидуальные варианты нелинейных уравнений (см. подраздел 3.6).

Порядок выполнения лабораторной работы №4:

- 1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям применимости метода).
- 2) Составить подпрограмму - функцию вычисления функции $f(x)$, предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.
- 3) Составить главную программу, вычисляющую корень уравнения $f(x) = 0$ и содержащую обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.
- 4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

В подразделе 3.7 приводится текст программы - функции HORDA, предназначенной для решения уравнения $f(x) = 0$ методом хорд.

Вариант 18: $f(x) = e^{\frac{1}{x^2}} - \ln x$

Отделение корней уравнения.

Рассмотрим функцию $f(x) = e^{\frac{1}{x^2}} - \ln x$ и её поведение. Она определена и непрерывна на $(0, +\infty)$. Её производная равна:

$$f'(x) = -\frac{2e^{\frac{1}{x^2}} + x^2}{x^3}$$

Функция строго убывает на всей области определения $(0; +\infty)$

При $x \rightarrow 0^+$: $f(x) \rightarrow +\infty$.

При $x \rightarrow +\infty$: $f(x) \rightarrow -\infty$.

Функция непрерывна на $(0, +\infty)$ и меняет знак с $+\infty$ на $-\infty$. По теореме Больцано-Коши корень существует. Так как функция строго убывает, корень единственный.

Локализуем корень:

Найдем значения функции в некоторых точках, чтобы определить отрезок для метода бисекции:

Локализуем корень:

При $x = 1$: $f(1) = e^1 - \ln 1 = e - 0 \approx 2.718 > 0$

При $x = 3$: $f(3) = e^{(1/9)} - \ln 3 \approx 1.117 - 1.099 \approx 0.018 > 0$

При $x = 4$: $f(4) = e^{(1/16)} - \ln 4 \approx 1.064 - 1.386 \approx -0.322 < 0$

Для метода бисекции будет взят отрезок $[3, 4]$, так как на его концах функция принимает значения разных знаков.

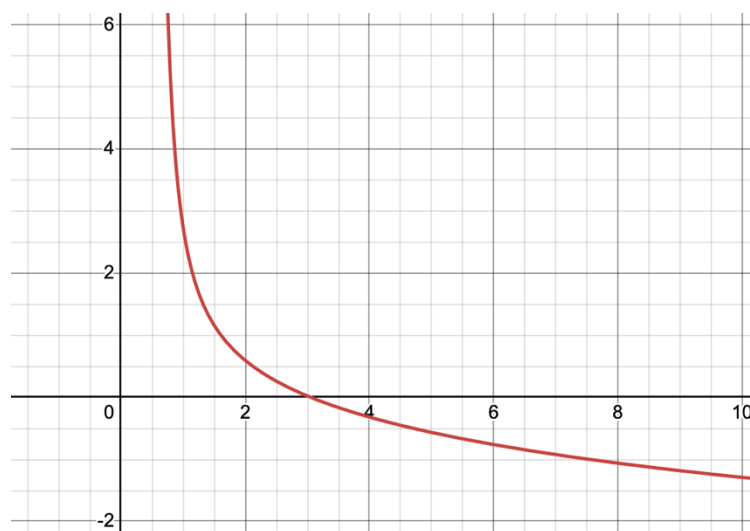


Рисунок 2 – График $f(x)$

Разработка программы.

Программа, реализующая метод хорд, была написана на языке программирования Python.

Исследование скорости сходимости.

Для вычисления ошибки и скорости сходимости в *horda* возвращался значение x каждой итерации. Скорость сходимости для метода хорд линейная и вычисляется по формуле $C \approx \frac{|x_{n+1}-x^*|}{|x_n-x^*|}$

Ниже приведен график скорости сходимости на каждой итерации (см. рис. 2).

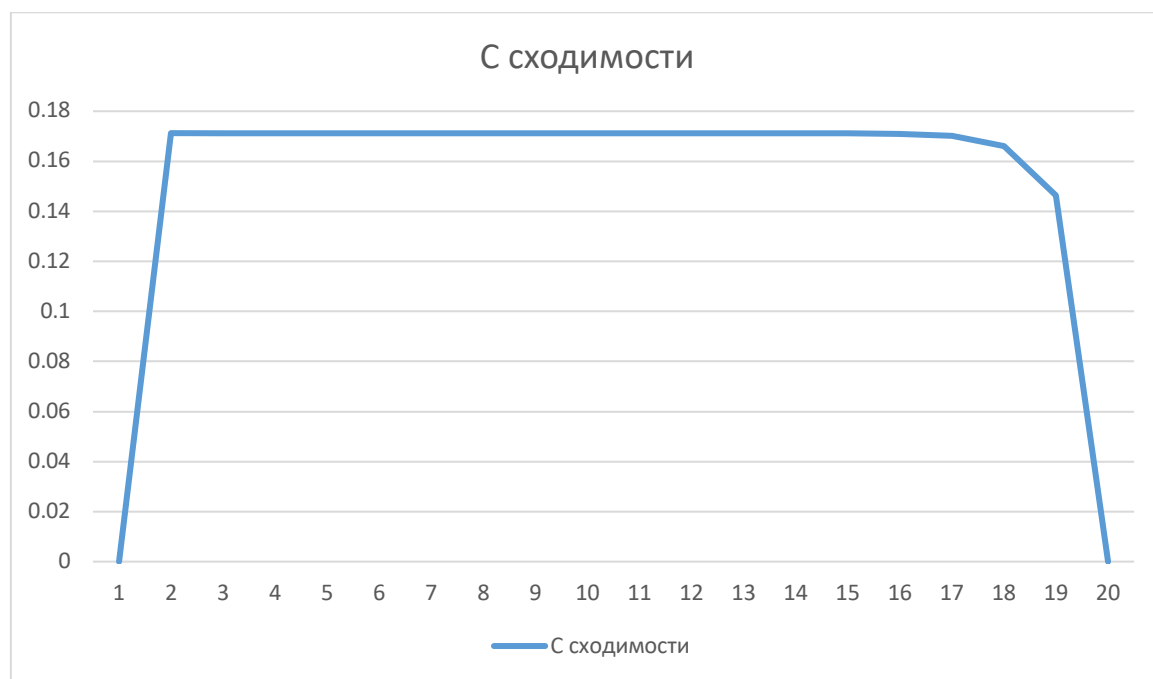


Рисунок 2 – Зависимость числа итераций от Eps

Эталонный корень: 3.045936104786175

Исследование скорости сходимости:

Итер.	Корень	Ошибка	C сходимости
1	3.055492836088	9.557e-03	0.00000
2	3.046039447021	1.033e-04	0.01081
3	3.045937223295	1.119e-06	0.01082
4	3.045936116892	1.211e-08	0.01082
5	3.045936104917	1.310e-10	0.01082
6	3.045936104788	1.418e-12	0.01082
7	3.045936104786	1.554e-14	0.01096
8	3.045936104786	0.000e+00	0.00000

Из графика видно, что полученное на практике значение близко к теоретической оценке количества итераций.

Тестирование.

В табл. 1 приведены результаты работы программы для различных значений эпсилон.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные
4.	0.1	3.1
5.	0.01	3.06
6.	0.000001	3.045937

Чувствительность метода к ошибкам в исходных данных.

Исследование обусловленности (влияние delta на корень):

Точность Eps = 1e-03:

Delta	Корень	Ошибка	
1e-01	3.00000000000000	4.594e-02	
1e-02	3.039215686275	6.720e-03	
1e-03	3.046028305495	9.220e-05	
1e-04	3.045985046267	4.894e-05	
1e-05	3.046032369453	9.626e-05	
1e-06	3.046040382551	1.043e-04	

Точность Eps = 1e-06:

Delta	Корень	Ошибка	
1e-01	3.00000000000000	4.594e-02	
1e-02	3.039215686275	6.720e-03	
1e-03	3.046028305495	9.220e-05	
1e-04	3.045985046267	4.894e-05	
1e-05	3.045935203502	9.013e-07	

| 1e-06 | 3.045935905586 | 1.992e-07 |

Выводы.

В результате выполнения лабораторной работы был исследован метод бисекции. Была написана программа на языке Python, реализующая данный метод. Была исследована скорость сходимости. Полученная практическая оценка скорости сходимости сошлась с теоретической.

Также была исследована чувствительность метода хорд к ошибкам во входных данных. Было установлено, что метод устойчив к ошибкам в исходных данных и выдает контролируемо малую погрешность в выходных данных

Цель работы.

Изучение метода Ньютона для решения нелинейных уравнений, написание программы, реализующей данный метод, исследование скорости сходимости (квадратичной) и обусловленности метода.

Задание.

В лабораторной работе № 5 предлагается, используя программы-функции NEWTON и ROUND из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения с заданной точностью Eps методом Ньютона, исследовать скорость сходимости и обусловленность метода.

Для данной работы вид функции задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения лабораторной работы №5.

1) Графически или аналитически отделить корень уравнения (т.е. найти отрезки [Left, Right], на котором функция удовлетворяет условиям сходимости метода Ньютона).

2) Составить подпрограммы - функции вычисления „, предусмотрев округление их значений с заданной точностью Delta.

3) Составить головную программу, вычисляющую корень уравнения и содержащую обращение к подпрограммам „, (x), Round, NEWTON и индикацию результатов.

4) Выбрать начальное приближение корня x_0 из $[Left, Right]$ так, чтобы >0 .!!!!!!

5) Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Для приближенного вычисления корней уравнения методом Ньютона предназначена программа - функция NEWTON, текст которой представлен в подразделе 3.7.

Цель работы.

Изучение метода простых итераций для решения нелинейных уравнений, написание программы, реализующей данный метод, исследование скорости сходимости и обусловленности метода.

Задание.

В лабораторной работе № 6 предлагается, используя программы-функции ITER и Round из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения с заданной точностью Eps методом итераций, исследовать скорость сходимости и обусловленность метода.

Для данной работы вид функции задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения лабораторной работы №6 должен быть следующим.

1) Графически или аналитически отделить корень уравнения .

2) Преобразовать уравнение к виду так, чтобы в некоторой окрестности $[Left, Right]$ корня производная удовлетворяла условию q . При этом следует иметь в виду, что чем меньше величина q , тем быстрее последовательные приближения сходятся к корню.

3) Выбрать начальное приближение, лежащее на $[Left, Right]$.

4) Составить подпрограмму для вычисления значений δ , предусмотрев округление вычисленных значений с точностью Delta.

5) Составить главную программу, вычисляющую корень уравнения и содержащую обращение к программам δ и ITER и индикацию результатов.

6) Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

Текст программы - функции ITER, позволяющей вычислять корни уравнения $x=$ для любой функции, которая удовлетворяет достаточным условиям сходимости метода, приводится ниже.

Вывод.

В результате выполнения лабораторной работы был исследован методы хорд, бисекции, Ньютона и простых итераций. Была написана программа на языке Python, реализующая данный метод. В ходе работы была подтверждена теоретическая зависимость числа итераций от требуемой точности, а также установлена устойчивость методов к ошибкам во входных данных.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab1.1.py

```
import math

def func(x):
    """Вычисление значения функции  $f(x) = e^{(1/x^2)} - \ln(x)$ """
    return math.exp(1 / (x**2)) - math.log(x)

def round_value(x, delta):
    """Функция округления с заданной точностью delta"""
    if delta == 0:
        return x
    return delta * int(x / delta + (0.5 if x > 0 else -0.5))

def bisection(f, left, right, eps):
    """Реализация метода бисекции с подсчётом итераций"""
    f_left = f(left)
    f_right = f(right)
    iterations = 0
    e = abs(eps) * 2.0

    if f_left * f_right > 0:
        raise ValueError("Функция не меняет знак на концах интервала")

    if f_left == 0:
        return left, 0
    if f_right == 0:
        return right, 0

    while (right - left) >= e:
        mid = (left + right) * 0.5
        f_mid = f(mid)

        if f_mid == 0:
            return mid, iterations

        # Если знаки f(left) и f(mid) разные, корень
        # слева
        if f_mid * f_left < 0:
            right = mid
        else:
            left = mid
            f_left = f_mid

        iterations += 1
    if iterations >= 10000:
```

```

        print("Достигнут лимит итераций")
        break

    return (left + right) * 0.5, iterations

def main():
    # Отрезок локализации для  $f(x) = e^{(1/x^2)} - \ln(x)$ 
    left, right = 3.0, 4.0

    print("Введите требуемую точность (например
    p, 0.001):")
    try:
        input_str = input().strip()
        eps = float(input_str)
        # Определяем количество знаков после з
    а пятой для вывода
        precision = len(input_str.split('.')[1]) if '.' in input_str
    else 2

    except ValueError:
        eps = 0.001
        precision = 3

    # Основной расчет
    root, _ = bisect(func, left, right, eps)
    print(f"\nНайденный корень: {root:.{precision}f}")

    print("\n" + "="*45)
    print(f"| {'Eps':<8} | {'Корень':<18} | {'Итер.':<6} |")
    print("-" * 45)

    # Тестирование для различных значений то
    чности
    for p in range(1, 7):
        current_eps = 0.1**p
        root, iters = bisect(func, left, right, current_eps)
        print(f"| {current_eps:<8.{p}f} | {root:<18.10f} | {iters:<6}
    |")

    # Исследование чувствительности к delta
    print("\nИсследование чувствительности к ок
    руглению (delta):")
    test_eps = 0.0001
    print(f"(при фиксированном Eps = {test_eps})")
    print("-" * 35)
    print(f"| {'Delta':<8} | {'Корень':<18} |")
    print("-" * 35)

    for p in range(1, 7):
        delta = 0.1**p
        # Создаем лямбда-функцию с округлением
    значения
        rounded_func = lambda x: round_value(func(x), delta)
        try:
            root, _ = bisect(rounded_func, left, right, test_eps)
            print(f"| {delta:<8.{p}f} | {root:<18.10f} |")

```

```

except Exception as e:
    print(f"| {delta:<8.{p}f} | Ошибка: корень вне
диапозона |")

```

```

if __name__ == "__main__":
    main()

```

Название файла: lab1.2.py

```

import math
import sys

def func(x):
    """Вычисление значения функции  $f(x) = e^{(1/x^2)} - \ln(x)$ """
    try:
        return math.exp(1 / (x**2)) - math.log(x)
    except OverflowError:
        return float('inf')

def round_custom(x, delta):
    """Функция округления с заданной точностью
    ю delta"""
    if delta <= 0:
        return x
    return round(x / delta) * delta

def horda(f, left, right, eps):
    """Реализация метода хорд с сохранением шагов"""
    f_left = f(left)
    f_right = f(right)
    steps = []

    if f_left * f_right > 0:
        raise ValueError("Функция не меняет знак на концах интервала")

    if abs(f_left) < 1e-18: return left, steps
    if abs(f_right) < 1e-18: return right, steps

    while True:
        # Вычисление точки пересечения хорды с
        осью OX
        x = x1 - f(x1) * (x2 - x1) / (f(x2) - f(x1))
        x = left - (right - left) * f_left / (f_right - f_left)
        y = f(x)
        steps.append(x)

        if abs(y) < eps or len(steps) >= 100:
            break

        # Сужаем интервал
        if y * f_left < 0:
            right = x
            f_right = y

```

```

        else:
            left = x
            f_left = y

    return x, steps

def main():
    # Параметры для  $f(x) = e^{(1/x^2)} - \ln(x)$ 
    left, right = 3.0, 4.0

    # Проверка режима (имитация --table через вв
од)
    print("Выберите режим: 1 - Одиночный расчет,
2 - Анализ таблицы")
    mode = input("Введите 1 или 2: ").strip()

    if mode == "1":
        print("Введите точность (например, 0.0001):")
        inp = input().strip()
        eps = float(inp)
        precision = len(inp.split('.')[1]) if '.' in inp else 2

        root, steps = horda(func, left, right, eps)
        print(f"\nКорень: {root:.{precision}f}")
        print(f"Итераций: {len(steps)}")
        return

    # РЕЖИМ ТАБЛИЦ (Анализ сходимости и обусл
овленности)

    # 1. Эталонный корень
    root_true, steps_full = horda(func, left, right, 1e-15)
    print(f"\nЭталонный корень: {root_true:.15f}")

    # 2. Исследование скорости сходимости
    print("\nИсследование скорости сходимости:")
    print(f"| {'Итер.':<10} | {'Корень':<15} | {'Ошибка
':<10} | {'Сходимости':<13} |")
    print("-" * 60)

    for i in range(len(steps_full)):
        current_x = steps_full[i]
        error = abs(current_x - root_true)

        # Константа сходимости  $C = e_n / e_{n-1}$ 
        conv_c = 0.0
        if i > 0:
            prev_error = abs(steps_full[i-1] - root_true)
            if prev_error != 0:
                conv_c = error / prev_error

        print(f"| {i+1:<10} | {current_x:<15.12f} | {error:<10.3e} |
{conv_c:<13.5f} |")

```

```

# 3. Исследование обусловленности (чувств
ительность к delta)
print("\nИсследование обусловленности (влия
ние delta на корень):")
for p_eps in [3, 6]:
    eps = 10**(-p_eps)
    print(f"\nТочность Eps = {eps:.0e}:")
    print(f"| {'Delta':<8} | {'Корень':<15} | {'Ошибка
':<10} |")
    print("-" * 40)

    for p_delta in range(1, 7):
        delta = 10**(-p_delta)
        # Оборачиваем функцию в округление
        rounded_f = lambda x: round_custom(func(x), delta)

        try:
            root_res, _ = horda(rounded_f, left, right, eps)
            err_res = abs(root_res - root_true)
            print(f"| {delta:<8.0e} | {root_res:<15.12f} |
{err_res:<10.3e} |")
        except:
            print(f"| {delta:<8.0e} | Ошибка поиска |")

if __name__ == "__main__":
    main()

```

Название файла: lab1.3.py

```

import math
import matplotlib.pyplot as plt

```

```

def function(x, delta=0.0):
    result = (math.log(x))**2 - 1.0/x
    if delta > 0:
        return round(result, delta)
    return result

```

```

#Производная функции
def f1(x, delta=0.0):
    """Производная f'(x) = (2*ln x)/x + 1/x^2"""
    result = (2.0 * math.log(x)) / x + 1.0/(x**2)
    if delta > 0:
        return round(result, delta)
    return result

```

```

#Функция округления
def round(x, delta):
    """
    Округление числа с точностью
    x - число для округления
    delta - точность округления
    """
    if delta <= 1e-9:

```

```

        print("Неверное задание точности округле
ния: delta должно быть > 1e-9")
        return
    if x > 0.0:
        return delta * int((x / delta) + 0.5)
    else:
        return delta * int((x / delta) - 0.5)

#Метод Ньютона
def newton(x, eps, delta=0.0):
    """
        Решение уравнения f(x)=0 методом Ньютона (к
асательных)

        Параметры:
        x - начальное приближение корня
        eps - точность
        delta - точность округления (для исследован
ия обусловленности)

        Возвращает кортеж:
        x - найденный корень
        n - число итераций
    """
    n = 0

    if eps <= 0.0:
        print("Неверное задание точности: eps долж
но быть > 0")
        return None, n

    while True:
        y = function(x, delta) # вычисление f(x)
        if y == 0.0: # если нашли точный корень
            return x, n

        y1 = f1(x, delta) # вычисление производной
f'(x)
        if y1 == 0.0: # проверка, что производная н
е ноль
            print("Производная обратилась в ноль")
            return None, n

        dx = y / y1 # вычисление приращения
        x = x - dx # новое приближение
        n += 1 # счетчик итераций

        if abs(dx) <= eps: # проверка условия оконча
ния
            return x, n

def main():
    try:

```

```

        x0 = float(input("Н а ч а л ь н о е   п р и б л и ж е н и е   x0   (н
а п р и м е р , 2.0)   в   о т р е з к е   л о к а л и з а ц и и :   "))
        eps = float(input("Т о ч н о с т ь   Eps   (о т 0.1   д о 0.000001):
"))
    except ValueError:
        print("О ш и б к а :   в в е д и т е   к о р р е к т н ы е   ч и с л а ")
        return

    if eps <= 0:
        print("О ш и б к а :   т о ч н о с т ь   д о л ж н а   б ы т ь   п о л о
ж и т е л ь н о й ")
        return

    # П о и с к   к о р н я
    x_root, iterations = newton(x0, eps)

    if x_root is None:
        return

    # В ы в о д   р е з у л ь т а т о в
    print(f"\nР Е З У Л Ь Т А Т Ы:")
    print(f"К о р е н ь :   x = {x_root:.8f}")
    print(f"Ч и с л о   и т е р а ц и й :   {iterations}")
    print(f"f(x) = {function(x_root):.2e}")
    print(f"f'(x) = {f1(x_root):.2e}")

    # И с с л е д о в а н и е   1:   З а в и с и м о с т ь   о т   т о ч н о с т и
    print("И С С Л Е Д О В А Н И Е   1:   З а в и с и м о с т ь   ч и с л а   и т
е р а ц и й   о т   т о ч н о с т и   Eps")

    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]
    iter_counts = []

    print("\nEps\t\tИ т е р а ц и и\tК о р е н ь ")
    print("-" * 60)
    for eps_val in eps_values:
        x_val, iter_val = newton(x0, eps_val)
        iter_counts.append(iter_val)
        print(f"{eps_val:.6f}\t{iter_val}\t\t{x_val:.8f}")

    # И с с л е д о в а н и е   2:   О б у с л о в л е н н о с т ь   (в л и я н и е
о ш и б о к   в   д а н н ы х )
    print("И С С Л Е Д О В А Н И Е   2:   В л и я н и е   о ш и б о к   в   и с х о
д н ы х   д а н н ы х ")

    # Т о ч н ы й   к о р е н ь   с   в ы с о к о й   т о ч н о с т ь ю
    x_exact, _ = newton(x0, 1e-10)
    delta_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]

    print(f"\n Т о ч н ы й   к о р е н ь   ( б е з   о ш и б о к ) :
{x_exact:.8f}\n")
    print("Delta\t\tК о р е н ь \t\tП о г р е ш н о с т ь ")
    print("-" * 60)

```



```

x_values_delta = []
for delta_val in delta_values:
    x_val, _ = newton(x0, 1e-6, delta_val)
    x_values_delta.append(x_val)
    error = abs(x_val - x_exact)
    print(f"{delta_val:.6f}\t{x_val:.8f}\t{error:.2e}")

# Построение графиков
plt.figure(figsize=(12, 5))

# График 1: Зависимость числа итераций от
# точности
plt.subplot(1, 2, 1)
plt.plot(eps_values, iter_counts, 'bo-', linewidth=2,
markersize=8)
plt.xscale('log')
plt.xlabel('Точность Eps', fontsize=12)
plt.ylabel('Число итераций', fontsize=12)
plt.title('Метод Ньютона: зависимость от точ
ности', fontsize=14)
plt.grid(True, alpha=0.3)

# График 2: Влияние ошибок входных данных
plt.subplot(1, 2, 2)
errors = [abs(x - x_exact) for x in x_values_delta]
plt.plot(delta_values, errors, 'ro-', linewidth=2, markersize=8)
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Delta (ошибка входных данных)',
fontsize=12)
plt.ylabel('Погрешность корня', fontsize=12)
plt.title('Влияние ошибок входных данных',
fontsize=14)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

main()

```

Название файла: lab1.4.py

```

import math
import matplotlib.pyplot as plt

def function(x, delta=0.0):
    """Исходная функция  $f(x) = \ln^2(x) - 1/x$ """
    result = (math.log(x)) ** 2 - 1.0 / x
    if delta > 0:
        return round_result(result, delta)
    return result

# Функция для метода итераций
def phi(x, delta=0.0):

```

```

"""
Ф у н к ц и я  phi(x) = exp(sqrt(1/x))
П о л у ч е н а  и з  у р а в н е н и я :  ln^2 x = 1/x  ->  ln x =
sqrt(1/x)  ->  x = exp(sqrt(1/x))
"""
result = math.exp(math.sqrt(1.0 / x))
if delta > 0:
    return round_result(result, delta)
return result

# П р о и з в о д н а я  ф у н к ц и и  phi
def phi_derivative(x, delta=0.0):
    """П р о и з в о д н а я  phi'(x) = -phi(x) / (2 * x^(3/2))"""
    result = -phi(x, delta) / (2.0 * (x ** 1.5))
    if delta > 0:
        return round_result(result, delta)
    return result

# Ф у н к ц и я  о к р у г л е н и я
def round_result(x, delta):
    """
    О к р у г л е н и е  ч и с л а  с  т о ч н о с т ь ю  delta
    x - ч и с л о  д л я  о к р у г л е н и я
    delta - т о ч н о с т ь  о к р у г л е н и я
    """
    if delta <= 1e-9:
        print("Н е в е р н о е  з а д а н и е  т о ч н о с т и  о к р у г л е
н и я :  delta  д о л ж н о  б ы т ь  > 1e-9")
        return x
    if x > 0.0:
        return delta * int((x / delta) + 0.5)
    else:
        return delta * int((x / delta) - 0.5)

# М е т о д  п р о с т ы х  и т е р а ц и й
def iteration_method(x0, eps, delta=0.0):
    """
    Р е ш е н и е  у р а в н е н и я  x = phi(x)  м е т о д о м  п р о с т ы х
и т е р а ц и й
    У с л о в и е  о с т а н о в к и :  |x_n - x_{n-1}| < eps
    """
    if eps <= 0.0:
        print("Н е в е р н о е  з а д а н и е  т о ч н о с т и ")
        return None, 0

    n = 0
    x_prev = x0

    while True:
        x_next = phi(x_prev, delta)
        n += 1

```

```

        if abs(x_next - x_prev) < eps:
            return x_next, n

    x_prev = x_next

def check_convergence(a, b):
    """
    Проверка условия сходимости  $|\phi'(x)| < 1$  на
    интервале [a, b]
    """
    print("\nПРОВЕРКА УСЛОВИЯ СХОДИМОСТИ")
    print("-" * 50)
    print(f"Интервал: [{a}, {b}]")
    print("\nx\tphi'(x)\t|phi'(x)|")
    print("-" * 30)

    x_values = [2.0, 2.1, 2.2, 2.3, 2.4, 2.5]
    q_max = 0

    for x in x_values:
        der = phi_derivative(x)
        abs_der = abs(der)
        print(f"{x:.1f}\t{der:.4f}\t{abs_der:.4f}")
        if abs_der > q_max:
            q_max = abs_der

    print("-" * 30)
    print(f"max |phi'(x)| = {q_max:.4f}")

    if q_max < 1:
        print("Условие сходимости выполняется:  $|\phi'(x)| < 1$  на всем интервале")
        print(f"q = {q_max:.4f} - гарантирует линейную сходимость")
    else:
        print("X Условие сходимости НЕ выполняется!")

    return q_max

def main():
    print("МЕТОД ПРОСТЫХ ИТЕРАЦИЙ")
    print("=" * 60)

    # Интервал локализации корня
    a, b = 2.0, 2.5

    # Проверка условия сходимости
    q_max = check_convergence(a, b)

    print("\n" + "=" * 60)

    # Ввод данных

```

```

try:
    x0 = float(input("Н а ч а л ь н о е   п р и б л и ж е н и е   x0   (н
а п р и м е р , 2.0): "))
    eps = float(input("Т о ч н о с т ь   Eps   (о т 0.1 д о 0.000001):
"))
except ValueError:
    print("О ш и б к а :   в в е д и т е   к о р р е к т н ы е   ч и с л а ")
    return

if eps <= 0:
    print("О ш и б к а :   т о ч н о с т ь   д о л ж н а   б ы т ь   п о л о
ж и т е л ь н о й ")
    return

# П о и с к   к о р н я
print(f"\nП о и с к   к о р н я   с   н а ч а л ь н ы м   п р и б л и ж е н и
е м   x0 = {x0}...")
x_root, iterations = iteration_method(x0, eps)

if x_root is None:
    return

# В ы в о д   р е з у л ь т а т о в
print(f"\nР Е З У Л Ь Т А Т Ы:")
print(f"К о р е н ь :   x = {x_root:.8f}")
print(f"Ч и с л о   и т е р а ц и й :   {iterations}")
print(f"f(x) = {function(x_root):.2e}")
print(f"phi'(x) = {phi_derivative(x_root):.4f}")

# И с с л е д о в а н и е   1:   З а в и с и м о с т ь   о т   т о ч н о с т и
print("\n" + "=" * 60)
print("И С С Л Е Д О В А Н И Е   1:   З а в и с и м о с т ь   ч и с л а   и т
е р а ц и й   о т   т о ч н о с т и   Eps")
print("=" * 60)

eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]
iter_counts = []
root_values = []

print("\nEps\t\tИ т е р а ц и и \tК о р е н ь ")
print("-" * 60)
for eps_val in eps_values:
    x_val, iter_val = iteration_method(x0, eps_val)
    iter_counts.append(iter_val)
    root_values.append(x_val)
    print(f"{eps_val:.6f}\t{iter_val}\t\t{x_val:.8f}")

# И с с л е д о в а н и е   2:   О б у с л о в л е н н о с т ь   (в л и я н и
е   о ш и б о к   в   д а н н ы х )
print("\n" + "=" * 60)
print("И С С Л Е Д О В А Н И Е   2:   В л и я н и е   о ш и б о к   в   и с х о
д н ы х   д а н н ы х ")
print("=" * 60)

```

```

# Точный корень с высокой точностью
x_exact, _ = iteration_method(x0, 1e-10)
delta_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]

print(f"\nТочный корень (без ошибок):
{x_exact:.8f}\n")
print("Delta\t\tКорень\t\tПогрешность")
print("-" * 60)

x_values_delta = []
error_values = []

for delta_val in delta_values:
    x_val, iter_val = iteration_method(x0, 1e-4, delta_val)
    x_values_delta.append(x_val)
    error = abs(x_val - x_exact)
    error_values.append(error)
    print(f"{delta_val:.6f}\t{x_val:.8f}\t{error:.2e}")

# Построение графиков
plt.figure(figsize=(14, 5))

# График 1: Зависимость числа итераций от
точности
plt.subplot(1, 2, 1)
plt.plot(eps_values, iter_counts, 'bo-', linewidth=2,
markersize=8)
plt.xscale('log')
plt.xlabel('Точность Eps', fontsize=12)
plt.ylabel('Число итераций', fontsize=12)
plt.title('Метод простых итераций:\nзависимо
сть от точности', fontsize=14)
plt.grid(True, alpha=0.3)

# Подписи к точкам
for i, (eps_val, iter_val) in enumerate(zip(eps_values,
iter_counts)):
    plt.annotate(f'{iter_val}', (eps_val, iter_val),
textcoords="offset points", xytext=(0, 10),
ha='center')

# График 2: Влияние ошибок входных данных
plt.subplot(1, 2, 2)
plt.plot(delta_values, error_values, 'ro-', linewidth=2,
markersize=8)
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Delta (ошибка входных данных)',
fontsize=12)
plt.ylabel('Погрешность корня', fontsize=12)
plt.title('Влияние ошибок входных данных',
fontsize=14)
plt.grid(True, alpha=0.3)

```

```
plt.axhline(y=1e-4, color='g', linestyle='--', alpha=0.7,  
label='eps = 1e-4')  
plt.legend()  
  
plt.tight_layout()  
plt.show()  
  
main()
```